Table des matières

Cours 08 — Les codes 1

Cours 08 — Les codes

Cours 8/12 · La Brûlerie — Formation Data Engineering

Sources	Ingestion	RAW
d'où vient la donnée	aller la chercher, et la charger	la déposer, la ranger, la traiter
✓ ici		

À ouvrir après avoir essayé. Le code garde les commentaires numérotés du cours, pour comparer ligne par ligne. Les requêtes se tapent dans le notebook de DuckDB (`duckdb -ui`, lancé depuis `Projet_Brulerie`), une par cellule ; le script se crée dans la première fenêtre, par le Bloc-notes.

1.2 Les moments marqués d'un lot, à leur instant

```
-- 1. les lectures marquées d'un lot, chacune à son instant : le début du lot, plus sa seconde
SELECT c.seconde, CAST(l.debut AS TIMESTAMP) + CAST(c.seconde AS INTEGER) * INTERVAL 1 SECOND AS instant,
       c.temp_grain, c.temp_air, c.evenement
FROM entrepot.raw_courbes AS c
JOIN entrepot.raw_lots AS l USING (lot_id)
WHERE c.lot_id = 'L-2025-0214' AND c.evenement IS NOT NULL
ORDER BY CAST(c.seconde AS INTEGER);
```

2.1 La clé

```
-- 1. les couples (lot, seconde) qui reviennent plus d'une fois
SELECT lot_id, seconde, COUNT(*) AS fois
FROM entrepot.raw_courbes
GROUP BY lot_id, seconde
HAVING COUNT(*) > 1;
```

2.3 Le pas

```
-- 1. pour chaque lecture, l'écart avec la lecture précédente du même lot, dans l'ordre du temps
WITH pas AS (
    SELECT CAST(seconde AS INTEGER) - LAG(CAST(seconde AS INTEGER))
           OVER (PARTITION BY lot_id ORDER BY CAST(seconde AS INTEGER)) AS ecart
```


© 2026 iAlexMG. Tous droits réservés.

```
    FROM entrepot.raw_courbes
)
-- 2. combien de lectures pour chaque écart
SELECT ecart, COUNT(*) AS lectures
FROM pas
GROUP BY ecart
ORDER BY ecart;
```

2.4 Les sondes

```
-- 1. les lectures, puis celles où chaque sonde a répondu
SELECT COUNT(*) AS lectures, COUNT(temp_grain) AS grain, COUNT(temp_air) AS air
FROM entrepot.raw_courbes;
```

01_de/sql/argent_torrefaction.sql

```
-- 01_de/sql/argent_torrefaction.sql : une ligne par lot, tirée de ses lectures aux 10 secondes
CREATE OR REPLACE TABLE argent_lots AS
WITH courbes AS (
    -- 1. par lot, ce que les sondes ont vu : les lectures, le premier craquement, la sortie
    SELECT lot_id, COUNT(*) AS lectures,
           MAX(CAST(seconde AS INTEGER)) FILTER (WHERE evenement = 'PREMIER_CRAQUEMENT') AS craquement_s,
           MAX(CAST(seconde AS INTEGER)) FILTER (WHERE evenement = 'SORTIE') AS duree_s,
           MAX(CAST(temp_grain AS DECIMAL(5, 1))) FILTER (WHERE evenement = 'SORTIE') AS temp_sortie
    FROM raw_courbes GROUP BY lot_id
)
-- 2. le journal du torréfacteur, pour les seuls lots dont la courbe est arrivée
SELECT l.lot_id, CAST(l.debut AS TIMESTAMP) AS debut, l.profil, c.lectures, c.craquement_s,
       c.duree_s, c.temp_sortie,
       -- 3. le développement : la part de la torréfaction qui suit le premier craquement, en %
       CAST(100 * (c.duree_s - c.craquement_s) / c.duree_s AS DECIMAL(4, 1)) AS developpement_pct
FROM raw_lots AS l
JOIN courbes AS c USING (lot_id);

-- 4. les lots de l'argent, contre ceux que le journal connaît déjà
SELECT (SELECT COUNT(*) FROM argent_lots) AS argent, (SELECT COUNT(*) FROM raw_lots) AS journal;
```

5.1 Le témoin : l’argent contre le journal

```
-- 1. chaque lot de l'argent, joint à sa ligne du journal
SELECT COUNT(*) AS lots,
       -- 2. un écart : une mesure qui diffère du journal, ou une lecture qui manque à la courbe
```


© 2026 iAlexMG. Tous droits réservés.

```
       COUNT(*) FILTER (WHERE a.craquement_s <> CAST(l.premier_craquement_s AS INTEGER)
                           OR a.duree_s <> CAST(l.duree_s AS INTEGER)
                           OR a.temp_sortie <> CAST(l.temp_sortie AS DECIMAL(5, 1))
                           OR a.lectures <> a.duree_s / 10 + 1) AS ecarts
FROM entrepot.argent_lots AS a
JOIN entrepot.raw_lots AS l USING (lot_id);
```


© 2026 iAlexMG. Tous droits réservés.